BookLeaf Portal

APPROACH, TRADE-OFFS AND PRODUCTION EVOLUTION | SHASHANK

Problem and priorities

Authors need clear answers about publication, distribution and royalties; the operations team needs a manageable support queue with reliable context. I prioritized AI integration and code architecture, followed by authenticated API contracts and complete ticket workflows. The interface stays simple so reviewers can follow an author query through classification, a reviewed response and resolution.

Architecture and data

Next.js and TypeScript provide the author and operations workspaces. Presentation components, domain workflows, database repositories and AI providers have separate responsibilities. Server Components read through repositories directly; browser mutations use the versioned REST API. This avoids an extra HTTP round trip for server-rendered reads while retaining documented integration endpoints. Supabase supplies Auth, PostgreSQL and Realtime. Trusted role claims control operations access, and database Row Level Security independently enforces author ownership. Zod validates incoming API requests and structured model output; errors have consistent codes and request IDs.

The seed preserves the supplied ten authors and eighteen books, including both production titles and zero-royalty records. Each author has an isolated synthetic demo login. Sneha and Kavita expose production, zero earnings and pending payouts without modifying the sample data. Explicit loading, empty and zero-value states explain expected outcomes. Ticket and message pagination, screen-specific projections, authentication deduplication and Mumbai execution reduce avoidable work and latency.

AI strategy and failure handling

The ticket is saved before AI work starts. A provider-neutral service supports Gemini, local Ollama and manual review. Classification returns a category, priority, confidence and rationale; drafting uses relevant BookLeaf policy and trusted author/book context. Output schemas, token limits, timeouts and bounded retries constrain provider behavior. Untrusted ticket content is separated from policy instructions. Drafts require an administrator's review and send action. AI audit records capture provider/model, usage, timing and safe failure information; regeneration preserves a previous usable draft if the new attempt fails.

Manual mode is the repository default and incurs no model calls. The hosted audit log contains a successful Gemini classification and draft with token usage recorded; seeded demo entries are not counted as provider evidence. This verifies one execution, not classification accuracy across all categories or future provider availability.

Trade-offs, validation and production evolution

The assignment uses a single application and a small seeded database to keep setup and review practical. Checks include TypeScript, ESLint, a production build, 25 automated tests and authenticated ownership checks for all ten authors. These do not replace browser end-to-end, load or adversarial testing. API contracts and the current limitations are documented separately.

For production, move post-response AI work to a durable queue with idempotency, retries and recovery; add a representative labelled evaluation set for classification, priority, grounding and unsafe-output tests. Introduce per-user quotas and spending alerts, track review acceptance and escalation rates, and monitor request latency and failures. Replace shared demo credentials, add account provisioning and recovery, review retention and privacy requirements, and use an appropriate commercial privacy configuration or a controlled self-hosted model. Add tested backups, recovery drills, CI release gates and broader workflow tests before serving real authors. Expand billing and payout integrations only when authoritative accounting data is available.